CS525 – Formal Methods for System Verification

Course Project – Final Report

LLM-Driven SystemVerilog Assertion Generation for Formal Verification

Addressing Long-Context Understanding, Efficient Communication, and Structured Reasoning for Scalable Assertion Synthesis

Group 2 · Project 2

Pranjal	230101077
Dhruvkumar R Pansuriya	230101071
Shrish Uttarwar	230101108

Date: 23 April 2026

Indian Institute of Technology Guwahati

Department of Computer Science & Engineering

1. Abstract

SystemVerilog Assertions (SVAs) are essential for Formal Verification but notoriously difficult to manually write. While Large Language Models (LLMs) offer automation potential, they struggle with context limits, fragmented design understanding, and multi-step reasoning inefficiencies.

In this report for CS525 Project 2, we present a modular, **LLM-driven SVA generation** pipeline. Our approach extends recent structured synthesis methods to tackle five core challenges: *(i) long-context limits, (ii) agent memory, (iii) cloud communication efficiency, (iv) redundancy in multi-agent systems, and (v) feedback-driven refinement.*

By integrating structured specification and RTL representations with retrieval-driven context selection, prompt compression, and iterative refinement, our system delivers a more context-aware, cost-efficient, and robust assertion generation framework for practical formal verification workflows.

2. Introduction

Assertion-Based Verification (ABV) is the dominant paradigm for ensuring that VLSI designs behave according to specification. SVAs allow engineers to express properties such as *"whenever a request is asserted, an acknowledgement must follow within three clock cycles"*, which formal tools can exhaustively verify. The primary bottleneck, however, is that writing SVAs requires simultaneously understanding high-level specification documents (often hundreds of pages of natural language) and low-level RTL implementations (thousands of lines of code).

Existing LLM-based methods such as AssertLLM [4] rely heavily on either specification text or **RTL in isolation**. This leads to incomplete design understanding: specification-only approaches miss implementation-level details, while RTL-only approaches lack the intended behavior captured in natural language. Consequently, generated assertions are often syntactically correct but semantically inaccurate.

Recent approaches attempt to bridge this gap by decomposing the problem into intermediate representations and multi-stage pipelines. Our work focuses on addressing a key systems challenge:

> *How can an automated system synthesise information from ambiguous, multi-page natural-language specifications and complex RTL code to generate SVAs that are both syntactically correct and functionally meaningful — within the token and cost constraints of modern LLMs?*

Our project addresses this challenge through five specific sub-problems:

P1. Long-Context Limits — selecting a minimal, sufficient context subset.

P2. Agent Memory — accumulating and reusing verified knowledge across steps.

P3. Cloud Communication Efficiency — reducing API token and latency overhead.

P4. Redundancy in Multi-Agent Systems — minimizing duplicated computation and assertions.

P5. Feedback-Driven Refinement — improving outputs using verifier or critic feedback.

This report presents our final system design, implementation, and evaluation, demonstrating how these challenges can be jointly addressed in a unified LLM-driven verification pipeline.

3. Related Work

3.1 AssertLLM

AssertLLM [4] is one of the earliest end-to-end pipelines for automating SVA generation from natural-language specifications. It decomposes the task into three stages: structural specification extraction, signal mapping, and assertion synthesis. This staged design highlights the importance of intermediate representations when bridging natural language and RTL-level semantics.

However, a key limitation of AssertLLM is that it operates primarily on specification documents, without incorporating sufficient information from the RTL implementation. As a result, when specifications are ambiguous or incomplete, the generated assertions may appear plausible but fail under formal verification. This limitation motivates the need for approaches that jointly reason over both specification intent and implementation details.

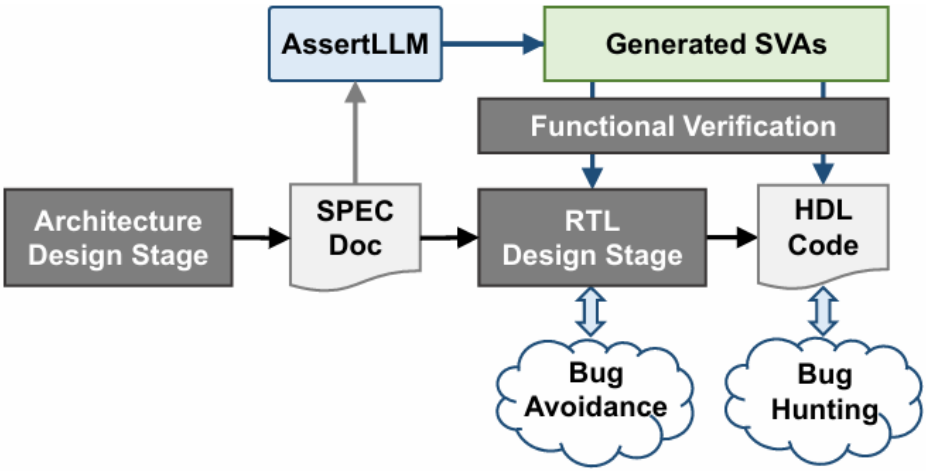


Figure 1: AssertLLM in VLSI design and verification flow. The system automatically generates SVAs from natural language specifications, supporting functional verification and bug detection.

3.1.1 Challenges in Understanding and Implementation

Despite its conceptual clarity, reproducing or extending AssertLLM presents several practical challenges:

- **Ambiguity in natural language specifications:** Translating informal and underspecified requirements into precise formal properties remains inherently difficult.
- **Limited implementation transparency:** The original work provides only a high-level pipeline description, with minimal details on prompt design, intermediate representations, model configurations, and post-processing. Critical steps such as structural extraction, signal alignment, and ambiguity resolution are not concretely specified, making faithful reproduction challenging.

4. Algorithm Deep Dive

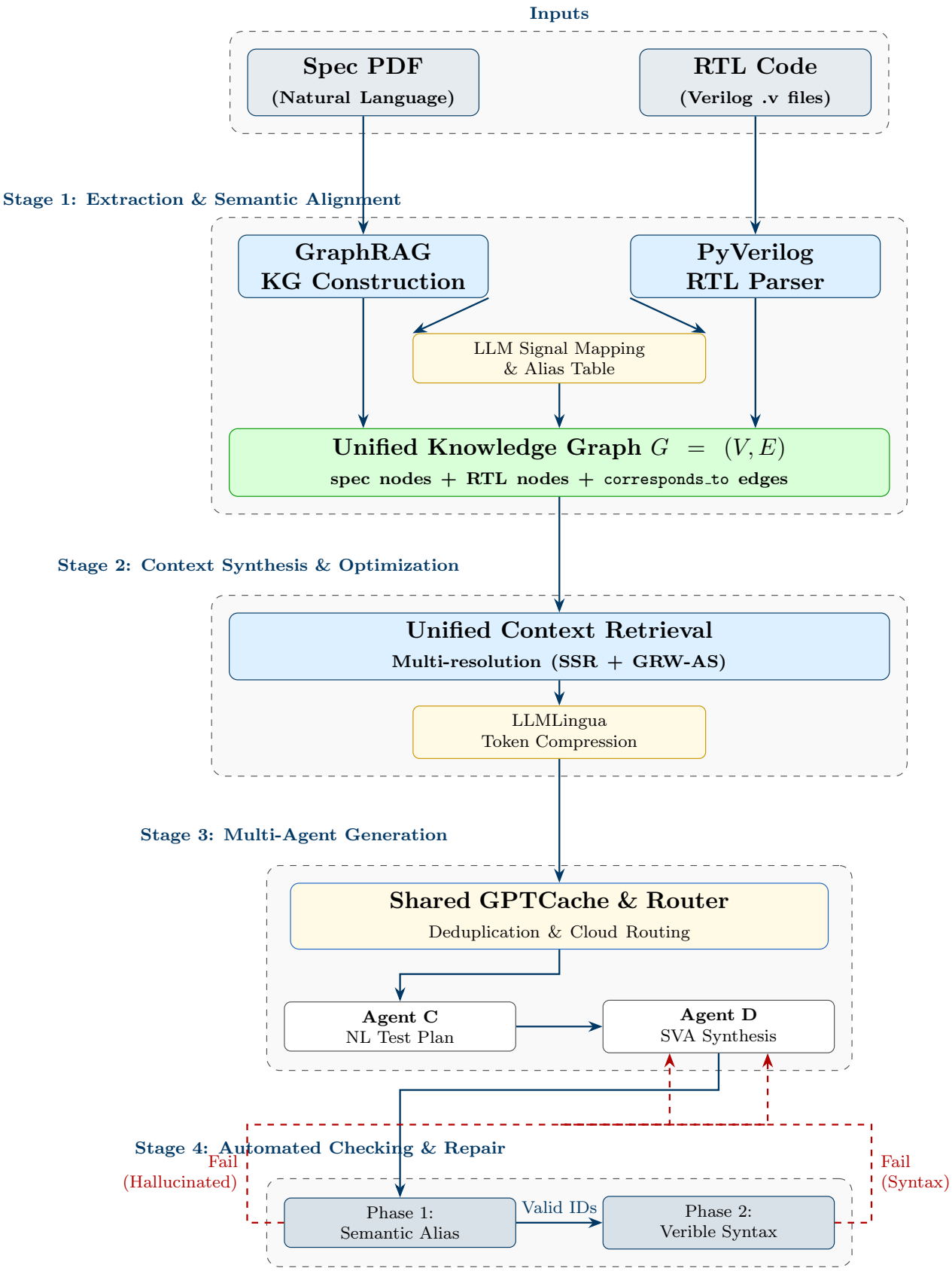


Figure 2: The Complete Phase II Pipeline Architecture. The system integrates intelligent signal mapping, aggressive prompt compression, multi-agent GPTCache routing, and a dual-phase semantic/syntactic validation loop to autonomously synthesize formal assertions.

4.1 Stage 1: Knowledge Graph Construction

4.1.1 Specification-Side: GraphRAG with Domain-Customised Schema

AssertionForge begins by constructing an initial KG G_0 from the specification document. It leverages Microsoft GraphRAG [5], a framework that (i) chunks source text, (ii) uses an LLM to extract entities and relationships, (iii) summarises node/edge descriptions, and (iv) assembles a graph. The critical novelty is that AssertionForge replaces GraphRAG's generic entity-extraction prompt with a hardware-specific schema $\Sigma = (E_t, R_t)$:

- **Entity types E_t:** Design Specification, Module, Submodule, Signal, Port, Register, FIFO, Clock, Interrupt, Protocol, Operation, Constraint/Rule, Address, FSM.
- **Relation types R_t:** `connectsTo`, `defines`, `triggers`, `dependsOn`, `generatesInterrupt`, `operatesAt`, `transmitsData`, `HasSignal`, `UsesClock`, `DescribesOperation`, *etc.*

Node attributes include entity type, description, source chunk ID, degree, and level. Edge attributes include weight and description. The schema is discovered semi-automatically: a small set of specification excerpts is shown to the LLM, which proposes entity/relation types; a human expert then merges redundant concepts (e.g., "Reset Signal" and "Reset").

4.1.2 RTL-Side: PyVerilog Parser and KG Refinement

The initial G_0 is then refined using structural and dataflow information extracted from RTL files via PyVerilog [6]:

1. **Module structure:** AST traversal identifies `ModuleDef` nodes, port declarations (direction, width), and module instantiations.
2. **Signals and assignments:** Both continuous (`assign`) and procedural (always-block) assignments are extracted. Each assignment records the LHS target signal and the RHS dependency set.
3. **FSM detection:** Always blocks with clock sensitivity lists containing `case` statements are tagged as FSM nodes; state variable names matching patterns like `state`, `st_`, `current_state` are identified.
4. **Dataflow analysis:** PyVerilog's `VerilogDataflowAnalyzer` builds a binding graph via `getBindings()`, adding dataflow edges between signals.

4.2 Signal Mapping: Specification–RTL Alignment

A critical component of our pipeline is the **signal mapping stage**, which aligns natural-language specification entities with RTL signals. This step bridges the semantic gap between high-level design intent and low-level implementation. In practice, specification text may loosely refer to a "system reset," while the RTL utilizes precise names like `nRST` or `rst_n`. Without this alignment, generated assertions frequently reference semantically correct but syntactically nonexistent signals, severing graph-based retrieval links and degrading SVA accuracy.

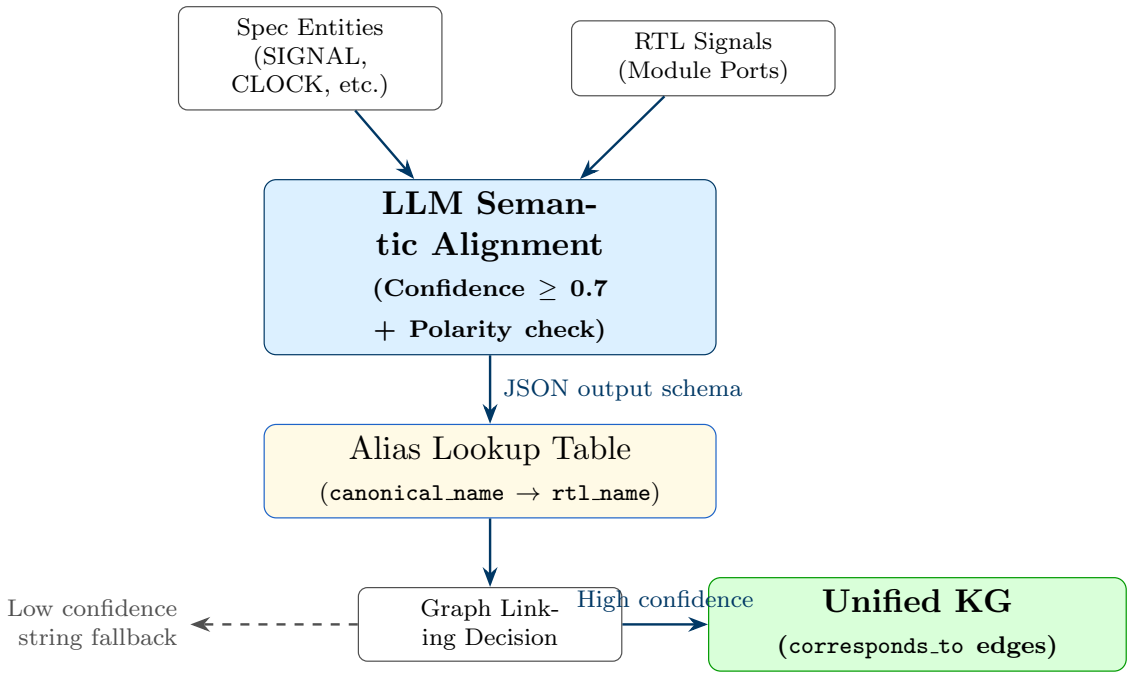


Figure 3: Signal Mapping Pipeline: Bridging specification intent with RTL syntax via LLM alignment and Knowledge Graph enrichment.

4.2.1 Methodology and Technical Implementation

The mapping stage is executed after the initial knowledge graph (KG) construction and before context retrieval. The implemented pipeline (`/src/signal_aligner.py`) operates through the following sequential flow:

1. Input Extraction & Preparation: We extract alignment inputs from both domains. Spec entities are isolated from KG node types (e.g., `SIGNAL`, `PORT`, `REGISTER`, `CLOCK`). Simultaneously, RTL module-level ports and internal signals are extracted via PyVerilog parsing and normalized to strip out comments and parser noise.

2. LLM-Based Semantic Alignment: An LLM performs semantic alignment between the spec entities and RTL signals. The model receives the extracted mentions, RTL signals (in `module.signal` format), and the surrounding local specification context. It outputs a strict JSON schema mapping canonical names to RTL modules, returning a confidence score and explicitly tracking metadata such as **active-low polarity**.

3. Alias Table Construction: The structured JSON outputs are converted into an **alias table**. This representation enables fast lookups from RTL signals to their canonical names, normalizes naming variations, and provides a structural basis for fuzzy fallback matching.

4. Confidence Gating & Graph Edge Creation: For each RTL signal, the system attempts to establish cross-domain links to specification nodes. We enforce a strict confidence threshold of **0.7**. If met, explicit `corresponds_to` edges are injected into the KG. Unresolved or low-confidence alignments fall back to a weaker string-based substring similarity to create `references` edges.

```
# Core Decision Logic for KG Integration
for each RTL signal:
    alignment = alias_lookup(signal)

    if alignment.confidence >= 0.7:
        add corresponds_to edges
    else:
        apply fuzzy matching fallback
```

4.2.2 Artifacts, Traceability, and Challenges

To ensure complete reproducibility and maintain explainability downstream, we treat diagnostics as first-class artifacts. The pipeline generates comprehensive logs per run, including prompt traces (`alignment_full_prompt.txt`), mapped lookup tables (`signal_aliases.json`), and explicit logging of edge creation (`signal_mapping_decisions.json`). Figure 4 illustrates a sample JSON output generated during this phase.


```
{
  "canonical_name": "peripheral_select",
  "spec_mentions": [
    "PERIPHERAL SELECT"
  ],
  "rtl_name": "sdc_top.psel",
  "rtl_module": "sdc_top",
  "active_low": false,
  "confidence": 1.0,
  "match_method": "llm_alignment",
  "reasoning": "The signal psel is a select signal, which matches the PERIPHERAL SELECT mention in the spec."
},
{
  "canonical_name": "transfer_enable",
  "spec_mentions": [
    "TRANSFER ENABLE"
  ],
  "rtl_name": "sdc_top.penable",
  "rtl_module": "sdc_top",
  "active_low": false,
  "confidence": 1.0,
  "match_method": "llm_alignment",
  "reasoning": "The signal penable is an enable signal for transfer, which matches the TRANSFER ENABLE mention in the spec."
},
```


Figure 4: JSON output excerpt from the LLM demonstrating semantic alignment, confidence scoring, and polarity tracking.

During implementation and testing, we observed several operational realities:

- **Graph connectivity dictates retrieval quality:** Missing edges directly lead to weak context synthesis, degrading downstream SVA accuracy.
- **Edge creation is brittle:** While the LLM often produces accurate mappings, converting them into hard graph edges strictly depends on successful string matching with spec node names.
- **Necessity of Fallbacks:** Without the fuzzy matching fallback, a large portion of signals would remain unlinked due to trivial naming mismatches.

4.2.3 Problems Addressed

This semantic bridge directly contributes to resolving multiple core challenges defined in our problem statement:

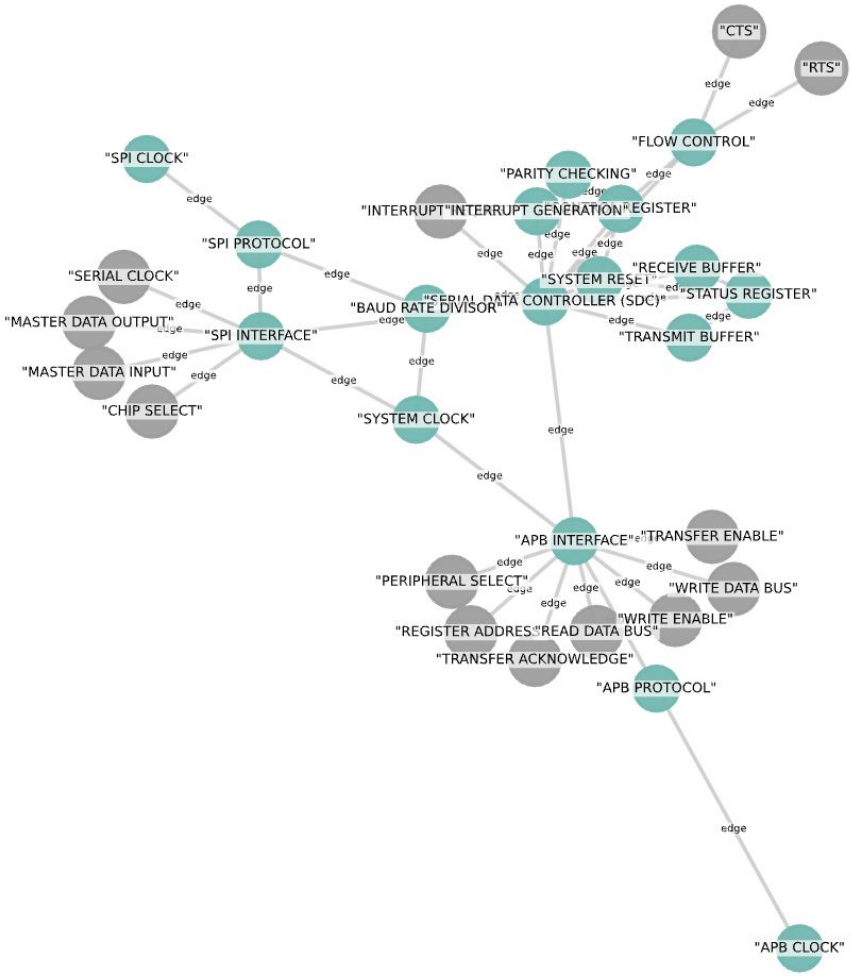


Figure 5: Knowledge graph visualization for a dummy spec and rtl.

- **P1 (Long Context):** Enables precise node-to-node linking, drastically reducing the need to inject large, irrelevant specification chunks into the prompt.
- **P2 (Agent Memory):** Encodes persistent, cross-domain knowledge directly within the KG for reuse in later stages.
- **P4 (Redundancy):** Avoids repeated textual inference by reusing statically mapped relationships across multiple assertion generations.

4.3 Stage 2: Multi-Resolution Context Synthesis

For each target architectural signal $v_i \in V$, the system constructs up to $B = 3$ distinct prompts by combining contexts from three generators:

(i) Global Summarisation. An LLM generates five complementary summaries from the full spec + RTL: a high-level design overview (3–5 sentences), an RTL architecture summary, a comprehensive per-signal summary, a design patterns summary (FSMs, handshake protocols,

pipelines), and a signal-specific description for v_i. These are generated once and cached across all signals.

(ii) Signal-Specific Retrieval (SSR). A hybrid retriever queries the specification PDF and RTL files using the signal name as the query term. It combines TF-IDF (sparse, keyword-sensitive) with Sentence Transformer embeddings (dense, semantic) at five granularity levels: 50, 100, 200, 800, 3200 tokens, with overlap ratios of 0.2 and 0.4 to address boundary fragmentation. The top 20 chunks per level are ranked by averaged cosine similarity.

(iii) Guided Random Walk with Adaptive Sampling (GRW-AS). GRW-AS is the most novel contribution of AssertionForge. Starting from the KG node corresponding to v_i, the algorithm performs 70 biased random walks with a 100-step budget each. At every step, a candidate next node c is selected with probability (Equations present in the paper). Walks terminate on reaching another architectural signal or exceeding the step budget. The resulting path is converted to a natural-language description ("signal A *drives* signal B, which *is part of* module C, which *involves* verification point D") and used as a structured context fragment.

LLM-as-Pruner. The union of SSR and GRW-AS contexts is passed to a pruner LLM that scores each context on relevance to v_i, information density, and diversity complementarity, retaining the highest-ranked subset to fit within the prompt token budget.

4.4 Stage 3: SVA Generation

The final pruned contexts are assembled into dynamic prompts and fed to GPT-4o in two substeps:

1. **NL Test Plan:** The LLM first generates natural-language descriptions of properties to verify (e.g., "Verify that `TX_EMPTY` asserts when the FIFO count reaches zero").
2. **SVA Synthesis:** Each NL plan is converted into a formal SVA. The prompt includes examples of correct SVA syntax to guide the model.

4.5 Why the KG Approach is Superior

The KG provides three capabilities that flat RAG cannot:

(i) **Cross-document traceability.** A single node can link a specification sentence to the RTL signal it describes, something no chunk-level retrieval can do reliably.

(ii) **Causal chain discovery.** GRW-AS surfaces paths like `TX_EMPTY` → `apb (module)` → `PRESETn` → `reset_behavior`, which reveal that `TX_EMPTY`'s reset is governed by an active-low global reset—information that would never appear in a single 200-token chunk.

(iii) **Compression resistance.** KG path descriptions are already dense and low-redundancy. Every token carries unique structural information, so they survive token-level compression better than verbose specification prose.

4.6 Knowledge-Graph as Robust Agent Memory (Addressing RQ1.4–RQ1.6)

The integration of autonomous agents in formal verification raises critical questions regarding how these agents store, retrieve, and maintain design knowledge over extended lifecycles. Flat vector databases (traditional RAG) fail when causal hardware logic is fragmented across semantic

chunks. By utilizing our unified Knowledge Graph (KG) as the primary memory structure, we directly address the challenges of memory fidelity, poisoning, and adaptability.

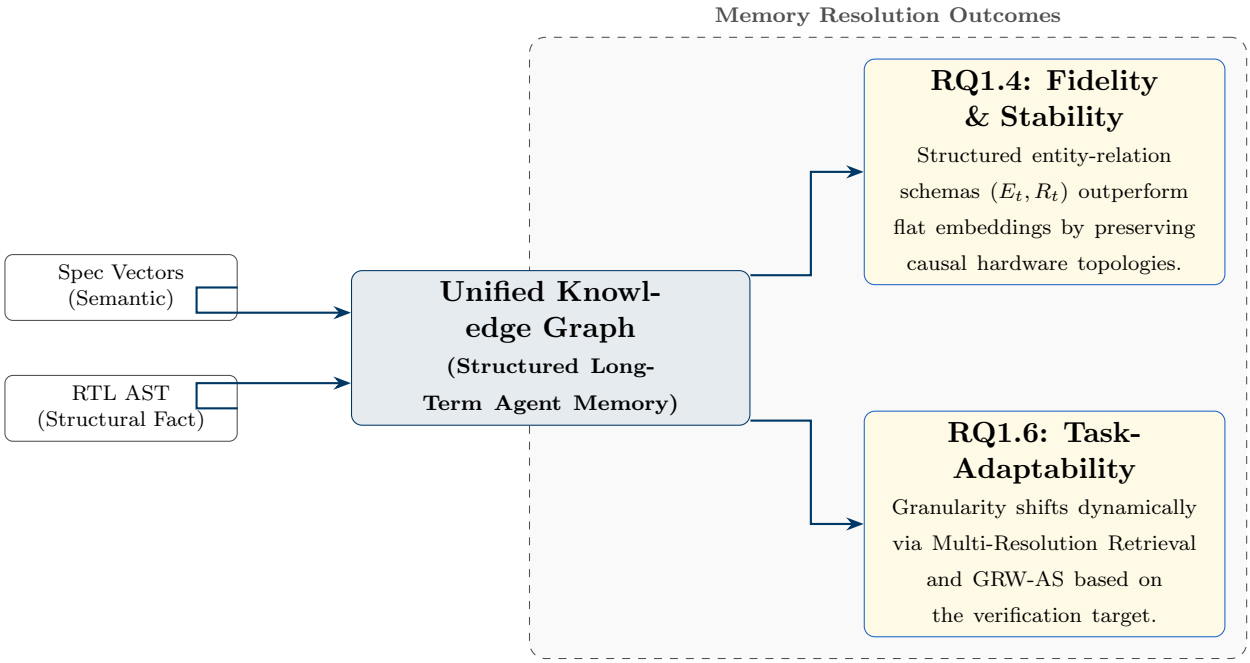


Figure 6: How the Unified Knowledge Graph solves the core challenges of agent memory representation, stability, and adaptive retrieval.

4.6.1 RQ1.4: Memory Representation and Retrieval Fidelity

What memory representation yields best retrieval fidelity and update stability?

Our implementation demonstrates that **structured facts (via KG schemas) combined with embeddings** significantly outperform pure text summaries or raw vector embeddings. Natural language specifications are highly verbose and semantically overlapping, causing traditional embedding similarity to retrieve redundant chunks while missing critical causal links (e.g., nested clock-domain crossings).

By extracting a hardware-specific schema (e.g., `Module`, `Signal`, `connectsTo`, `triggers`), the KG enforces a rigid, relational memory representation. Update stability is naturally maintained because new information acts as edge insertions or attribute updates on existing deterministic nodes, rather than diluting a vector space with overlapping text chunks.

4.6.2 RQ1.6: Task-Adaptive Memory Granularity

Can memory be task-adaptive, i.e., store different granularity depending on task type?

Yes. The KG inherently stores information at the atomic (node/edge) level, but retrieval is structurally adaptive based on the immediate verification task. We implement this through **Multi-Resolution Context Synthesis**:

- **Broad Debugging / Test Planning (Agent C):** The memory is accessed via coarse-grained Global Summarization and high-level Signal-Specific Retrieval (SSR) at 800+ token chunks.

nl_plans.txt U

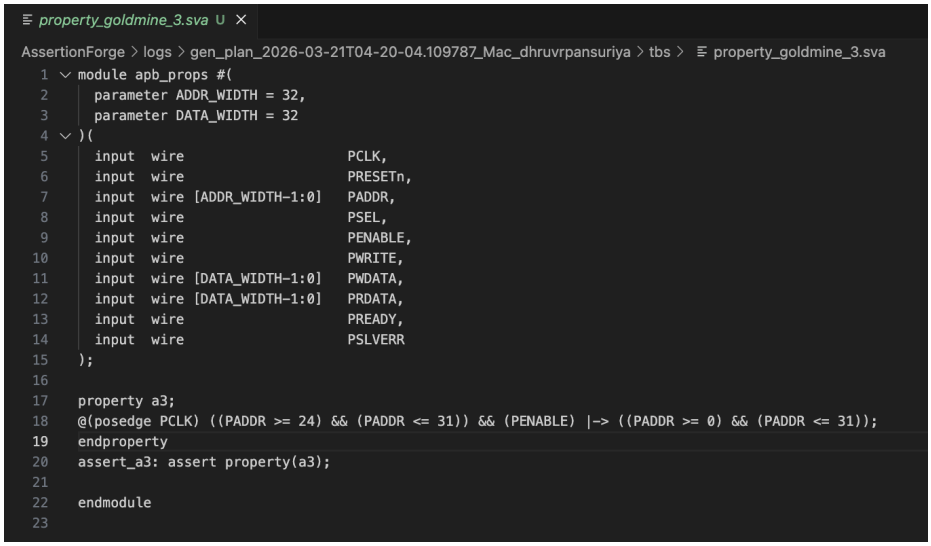
AssertionForge > logs > gen_plan_2026-03-21T04-20-04.109787_Mac_dhruvrpansuriya > nl_plans.txt

```
Signal PADDR:
Plan 1: PADDR: that when PADDR is within the range of 0 to 31, in the next cycle PADDR will remain within the valid address range.
Plan 2: PADDR: that the address signal PADDR has a value between 0 and 15, inclusive, 2 clock cycles after the reset signal PRESETn becomes deasserted, as indicated by the signal PCL
Plan 3: PADDR: that if the address signal PADDR is within the range of 16 to 23 inclusive, then in the subsequent cycles, PADDR must continue to be within the range of 0 to 31 inclus
Plan 4: PADDR: that the address signal PADDR is within the range 0 to 7 inclusive, starting from three clock cycles after the reset signal PRESETn becomes deasserted, as indicated by
Plan 5: PADDR: that when PADDR is within the range of 24 to 31, in the next cycle PADDR will remain within the valid address range, as controlled by the signal PENABLE.
Plan 6: PADDR: that the address signal PADDR has a value between 8 and 15, inclusive, 4 clock cycles after the reset signal PRESETn becomes deasserted, as indicated by the signal PCL
Plan 7: PADDR: that if the address signal PADDR is within the range of 0 to 7 inclusive, then in the subsequent cycles, PADDR must continue to be within the range of 0 to 15 inclusiv
Plan 8: PADDR: that the address signal PADDR is within the range 16 to 23 inclusive, starting from five clock cycles after the reset signal PRESETn becomes deasserted, as indicated b
Plan 9: PADDR: that when PADDR is within the range of 8 to 15, in the next cycle PADDR will remain within the valid address range, as controlled by the signal PENABLE and the select
Plan 10: PADDR: that the address signal PADDR has a value between 24 and 31, inclusive, 6 clock cycles after the reset signal PRESETn becomes deasserted, as indicated by the signal P

Signal PCLK:
Plan 11: PCLK: that when PCLK rises, if PENABLE is high and PSEL is high, then in the next clock cycle, PREADY will be high if the transfer is completed.
Plan 12: PCLK: that if PRESETn is low, then PCLK will not affect the state of PSLVERR until PRESETn becomes high.
Plan 13: PCLK: that when PCLK falls, if PWRITE is high, then in the next clock cycle, PRDATA will be invalid until PREADY becomes high.
Plan 14: PCLK: that after reset (PRESETn becomes high), PCLK will start generating clock edges, and PADDR will be stable after 2 clock cycles.
Plan 15: PCLK: that if PCLK is stable and PENABLE is high, then PSLVERR will be low if the transfer is successful.
Plan 16: PCLK: that when PCLK rises, if PSEL is low, then PENABLE will be ignored in the current clock cycle.
Plan 17: PCLK: that if PRESETn is high and PCLK is stable, then PADDR will be valid after 1 clock cycle.
Plan 18: PCLK: that when PCLK falls, if PENABLE is low, then PREADY will be low in the next clock cycle.
Plan 19: PCLK: that after 5 clock cycles of PCLK, if PWRITE is low, then PRDATA will be valid if PREADY is high.
Plan 20: PCLK: that if PCLK is not oscillating and PRESETn is high, then PSLVERR will be low after 3 clock cycles.

Signal PENABLE:
Plan 21: PENABLE: that when PENABLE is asserted and PSEL is high, PREADY will be set to 1 in the next clock cycle, as indicated by the clock signal PCLK.
Plan 22: PENABLE: that if PENABLE is low and PSEL is asserted, the state machine will move to the ACCESS phase in the next clock cycle, as determined by the clock signal PCLK.
Plan 23: PENABLE: that when PENABLE is asserted and PADDR is within the valid range, PSLVERR will be set to 0, indicating no error, as checked by the clock signal PCLK.
Plan 24: PENABLE: that if PENABLE is asserted and PWRITE is high, the write transfer will complete at the end of the ACCESS cycle, as indicated by the clock signal PCLK and the PREAD
Plan 25: PENABLE: that when PENABLE is asserted and PSEL is high, PRDATA will be valid in the next clock cycle, as determined by the clock signal PCLK and the PREADY signal.
Plan 26: PENABLE: that if PENABLE is low and PRESETn is asserted, the state machine will remain in the IDLE state, as checked by the clock signal PCLK.
Plan 27: PENABLE: that when PENABLE is asserted and PADDR is out of range, PSLVERR will be set to 1, indicating an error, as checked by the clock signal PCLK.
Plan 28: PENABLE: that if PENABLE is asserted and PSEL is low, the state machine will not move to the ACCESS phase, as determined by the clock signal PCLK.
Plan 29: PENABLE: that when PENABLE is asserted and PWRITE is low, the read transfer will complete at the end of the ACCESS cycle, as indicated by the clock signal PCLK and the PREAD
Plan 30: PENABLE: that if PENABLE is asserted and PSEL is high, PWDATA will be valid in the next clock cycle, as determined by the clock signal PCLK and the PREADY signal.
```

Figure 7: Generated natural-language test plans for a custom design.



property_goldmine_3.sva U

AssertionForge > logs > gen_plan_2026-03-21T04-20-04.109787_Mac_dhruvrpansuriya > tbs > property_goldmine_3.sva

```
module apb_props #(
    parameter ADDR_WIDTH = 32,
    parameter DATA_WIDTH = 32
)(
    input  wire                    PCLK,
    input  wire                    PRESETn,
    input  wire [ADDR_WIDTH-1:0]   PADDR,
    input  wire                    PSEL,
    input  wire                    PENABLE,
    input  wire                    PWRITE,
    input  wire [DATA_WIDTH-1:0]   PWDATA,
    input  wire [DATA_WIDTH-1:0]   PRDATA,
    input  wire                    PREADY,
    input  wire                    PSLVERR
  );

  property a3;
  @(posedge PCLK) ((PADDR >= 24) && (PADDR <= 31)) && (PENABLE) |-> ((PADDR >= 0) && (PADDR <= 31));
  endproperty
  assert_a3: assert property(a3);

  endmodule
```

Figure 8: SystemVerilog Assertions generated by the pipeline.

- **Precise SVA Synthesis (Agent D):** The system shifts to high-granularity, highly-dense memory retrieval. It utilizes **Guided Random Walk with Adaptive Sampling (GRW-AS)** to extract exact causal logic chains (e.g., `TX_EMPTY` → `apb` → `PRESETn`).

This adaptability is further refined by our **LLMLingua** integration, which applies aggressive compression to verbose specification memory while preserving dense RTL/structural memory uncompressed.

5. Addressing Core Challenges (P1–P5)

Based on our study of the system and the open problems in the PS, we propose the following extensions. Figure **??** shows how they integrate into the existing AssertionForge architecture.

5.1 The Missing Context Problem: A Root Cause Analysis

When a signal's Knowledge Graph (KG) neighbourhood is sparse, the generated SystemVerilog Assertions (SVAs) typically pass basic syntax checks but fail functionally because the LLM lacks the behavioral constraints necessary to write a correct property. Inspired by **ClarifyGPT's** mechanism for proactively detecting ambiguous prompts via consistency checks, we implement an early-stage "context gap" detection using a KG density threshold.

If a signal's KG degree falls below this threshold, we categorize the missing context into one of three distinct root causes:

Cause A: **KG Extraction Failure:** The signal's behavior is fully implemented in the RTL, but the GraphRAG pipeline failed to extract its relationships from the natural-language text, resulting in an artificially sparse KG neighborhood.

Cause B: **Specification Incompleteness:** The natural-language specification document is flawed and completely omits the behavioral rules or edge-case handling for the target signal. No retrieval method can find what is not documented.

Cause C: **Structural Blindness of Retrieval:** Traditional semantic or keyword search successfully retrieves text mentioning the signal by name, but entirely misses the underlying control flow (e.g., nested `if-else` conditions, FSM state transitions, or clock-domain crossings) that governs its actual hardware behavior.

AssertionForge's current Signal-Specific Retrieval (SSR) and Guided Random Walk (GRW-AS) pipelines are susceptible to all three failure modes. To resolve these gaps, we trigger two parallel, deterministic fallback approaches.

5.2 LLMLingua — Prompt Compression (P1 +P3)

LLMLingua [2] proposes a coarse-to-fine prompt compression framework that uses a small language model's perplexity scores to identify and remove low-information tokens from a prompt before sending it to an expensive black-box LLM. The system has three components: (1) a *Budget Controller* that allocates different compression ratios to instructions, demonstrations, and questions; (2) an *Iterative Token-Level Prompt Compression* (ITPC) algorithm that models inter-token dependencies during compression; and (3) a *Distribution Alignment* module that fine-tunes the small model to better approximate the target LLM's distribution. The paper reports up to 20× compression with less than 1.5 percentage-point performance drop. This is directly relevant to our P1 (long-context) and P3 (cloud efficiency) sub-problems.

5.2.1 Practical Challenge: Local Model Deployment

A key challenge encountered while working with LLMLingua is the requirement of running a small language model locally to compute perplexity scores for prompt compression. Although termed "small," these models still demand significant GPU/CPU memory, especially when processing long prompts or batching inputs. On resource-constrained machines, loading and running such models

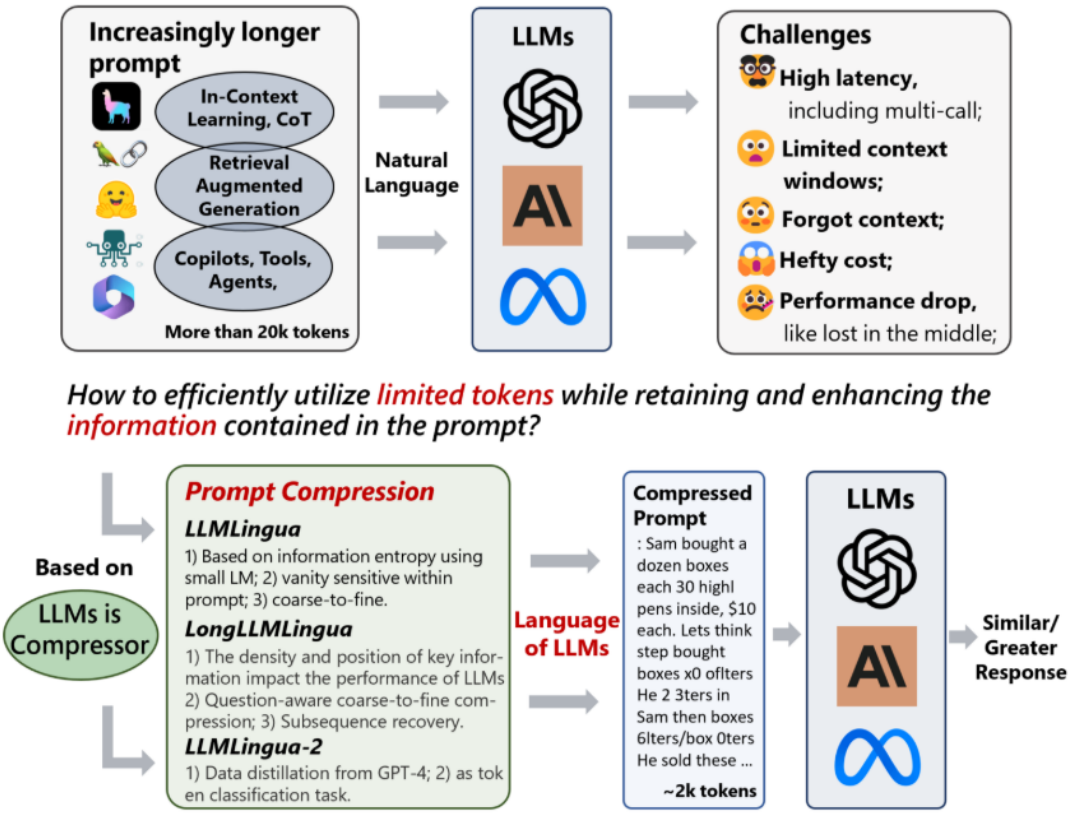


Figure 9: Why we need prompt compression.

can lead to memory exhaustion, slow inference, or the need for aggressive optimization techniques (e.g., quantization or offloading). This creates a practical barrier to adoption, particularly when the goal is to reduce cost and improve efficiency, as the overhead of maintaining a local model may offset some of the intended benefits.

Once the full context is assembled (the global summary, SSR chunks, GRW-AS paths, and our new RTL slices/STELLAR examples), the resulting prompt often exceeds optimal token limits, increasing API costs and diluting the LLM's attention. To address this, we introduce **LLMLingua** compression as a final preprocessing step before the prompt is sent to GPT-4o or the Groq API.

LLMLingua utilizes a smaller, highly efficient language model to calculate the perplexity of each token, stripping away low-information words while preserving semantic integrity. We utilize a dynamic budget controller tailored for hardware verification:

- **Low Compression Budget (Keep 70–80%):** Applied to GRW-AS KG paths, RTL backward slices, and STELLAR structural examples. These elements are highly dense, structurally rigid, and carry critical causal logic that breaks if over-compressed.
- **High Compression Budget (Keep 35–40%):** Applied to raw SSR text chunks retrieved from the specification PDF. Natural language spec documents are highly verbose, contain redundant phrasing, and recover extremely well under aggressive perplexity-based compression.

This targeted compression directly resolves the cloud communication efficiency bottleneck (P3) by drastically reducing tokens per API call, while simultaneously fulfilling the long-context limit objective (P1 RQ1.1) by ensuring only the minimal, sufficient context subset reaches the final

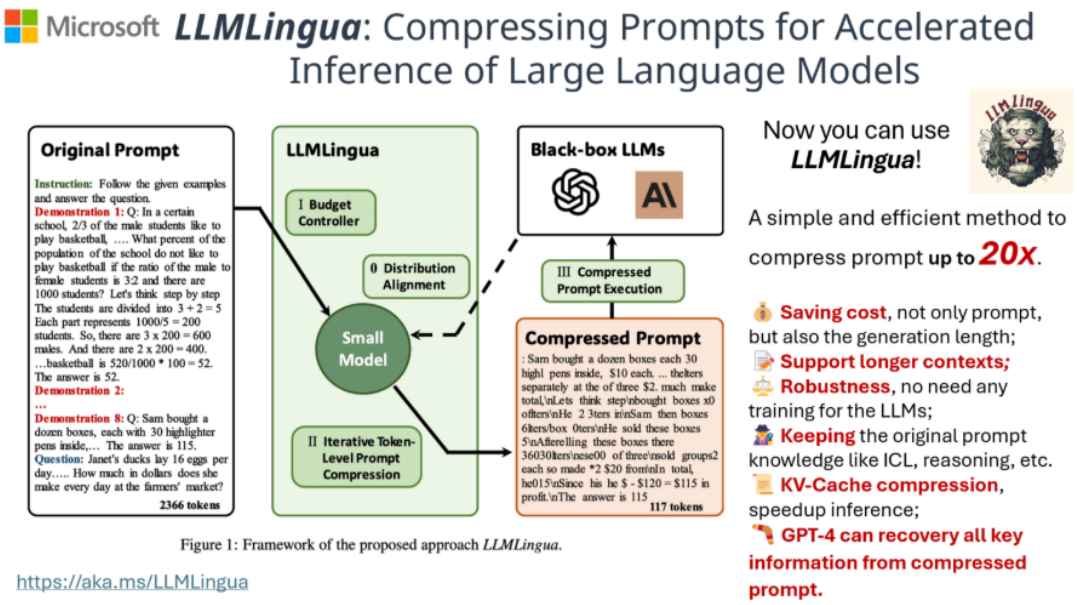


Figure 10: LLMLingua pipeline.

generation phase.

Problems Addressed

- **RQ1.1 (Minimal Context Selection):** LLM-Lingua implicitly approximates the optimal subset $C' \subset C$ by removing low-importance tokens and retaining high-salience information. This allows the system to pass a compressed yet relevant context to the LLM without explicitly performing hard retrieval.
- **RQ1.3 (Efficient Context Representation):** Prompt compression provides an alternative representation that is more compact than raw text while preserving semantic meaning. Compared to summaries, LLM-Lingua aims to retain token-level fidelity for critical components, making it suitable for reasoning-heavy tasks like assertion generation.
- **RQ1.7 (Minimizing Communication Overhead):** By reducing the total number of tokens sent in each request, LLM-Lingua directly lowers bandwidth usage and API costs in cloud-based inference. This is particularly important when large specifications or codebases must be transmitted repeatedly.
- **RQ1.8 (Safe Compression for Reasoning Tasks):** Unlike naive lossy summarization, LLM-Lingua performs importance-aware token pruning, which better preserves logical structure and dependencies required for formal reasoning. This makes it a safer compression strategy for verification tasks.

Key Benefits

- Significant reduction in token usage and inference cost
- Enables handling of larger contexts within fixed token limits
- Maintains higher fidelity compared to generic summarization
- Improves latency in cloud-based LLM calls

```
LLMLingua Token Reduction Summary
=================================

Overall Statistics:
  Tokens without LLMLingua : 4616
  Tokens with LLMLingua    : 3876
  Total Token Reduction    : 16.03%

Detailed Breakdown:
  Dynamic NL Generation:
    Origin: 2163, Compressed: 1807
  Static NL Generation:
    Origin: 0, Compressed: 0
  Dynamic SVA Generation:
    Origin: 2453, Compressed: 2069
  Static SVA Generation:
    Origin: 0, Compressed: 0
```

Figure 11: LLMLingua results.

Limitations and Considerations

- Compression may still remove subtle but important details, potentially affecting correctness in edge cases.
- Requires careful tuning of compression ratios depending on task sensitivity (e.g., verification vs general text tasks).
- Should be combined with retrieval or structured context selection for optimal performance.

5.3 Use of GPTCache for Prompt Caching and Mapping

To address challenges related to cloud communication efficiency and prompt reuse, we integrate **GPTCache** [13] as a caching layer within the LLM-based assertion generation pipeline.

Problems Addressed

- **RQ1.7 (Minimizing Information Transfer):** GPTCache reduces the need to repeatedly send large prompts and context to the cloud by storing previously computed responses. When a semantically similar query is encountered, the system retrieves the cached result instead of issuing a new API call, thereby significantly reducing bandwidth usage and latency.
- **RQ2.4 (Prompt Mapping / Similarity Detection):** GPTCache enables identification of "close enough" prompts using embedding-based similarity search. This allows the system to map new prompts to previously seen ones and reuse intermediate or final results safely when similarity exceeds a predefined threshold.

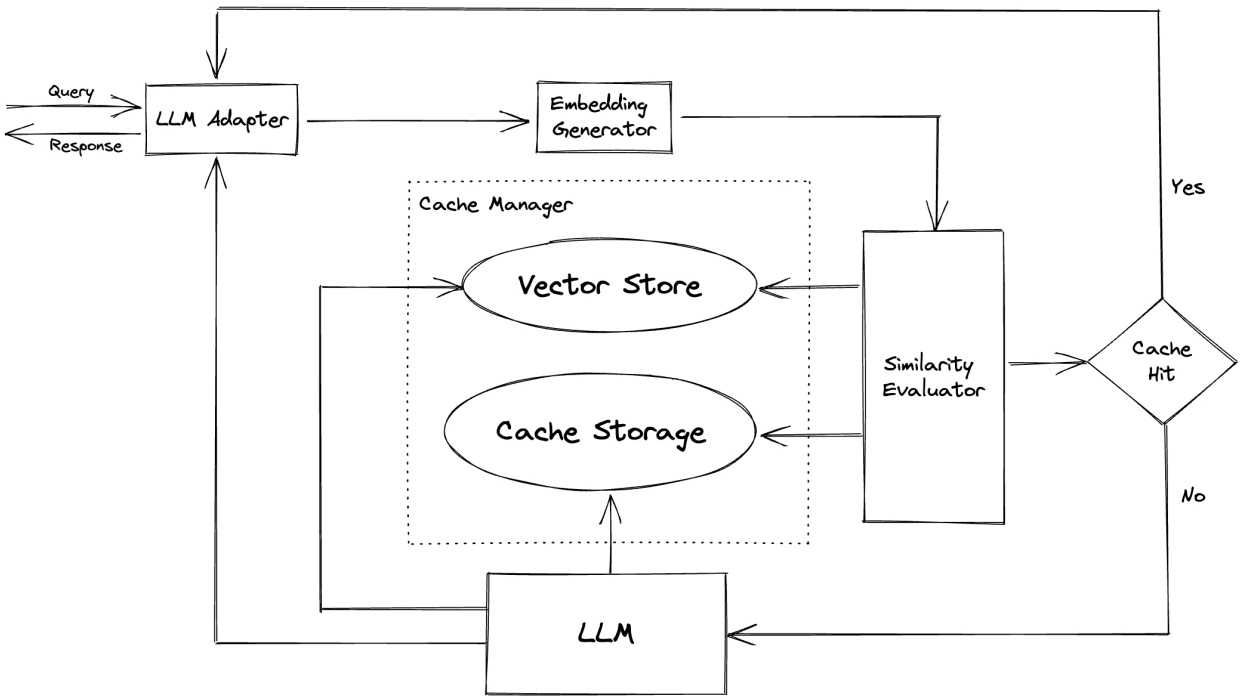


Figure 12: GPTCache pipeline.

- **RQ2.5 (Caching Granularity):** The caching mechanism operates effectively at the *prompt-level* and can be extended to *chunk-level* caching by storing responses for smaller context segments. This provides a flexible trade-off between reuse and specificity.
- **RQ2.2 (Early Redundancy Detection):** By intercepting repeated or highly similar queries before they are sent to the LLM, GPTCache prevents redundant computations across agents, reducing unnecessary token usage and API costs.
- **RQ1.8 (Safe Compression for Reasoning):** Instead of relying on lossy summarization, GPTCache preserves exact or high-fidelity responses. This ensures that reasoning-heavy outputs (e.g., formal assertions) are reused without degradation in correctness.

Key Benefits

- Reduced API calls and operational cost
- Lower latency due to cache hits
- Improved scalability in multi-agent settings
- Implicit prompt deduplication and reuse

Limitations and Considerations

- Cache validation is necessary to avoid propagation of incorrect or outdated responses (RQ2.6).
- Similarity thresholds must be carefully tuned to balance reuse and correctness.
- Cached responses may become stale if underlying context or specifications change.

5.4 Multiagent Orchestration with GPTCache

Our proposed end-to-end design for Phase II introduces a multiagent architecture coupled with a centralized semantic caching layer. Generating high-quality SVAs via automated pipelines requires issuing highly similar prompts repeatedly—such as during iterative syntax repairs, context synthesis, and requirement grounding. Routing all these overlapping requests directly to cloud models results in extreme latency and unacceptable API cost overheads.

To improve assertion quality through task specialization while maintaining cloud efficiency, we decompose the reasoning process into specialized agents that all communicate through a unified **GPTCache** mechanism.

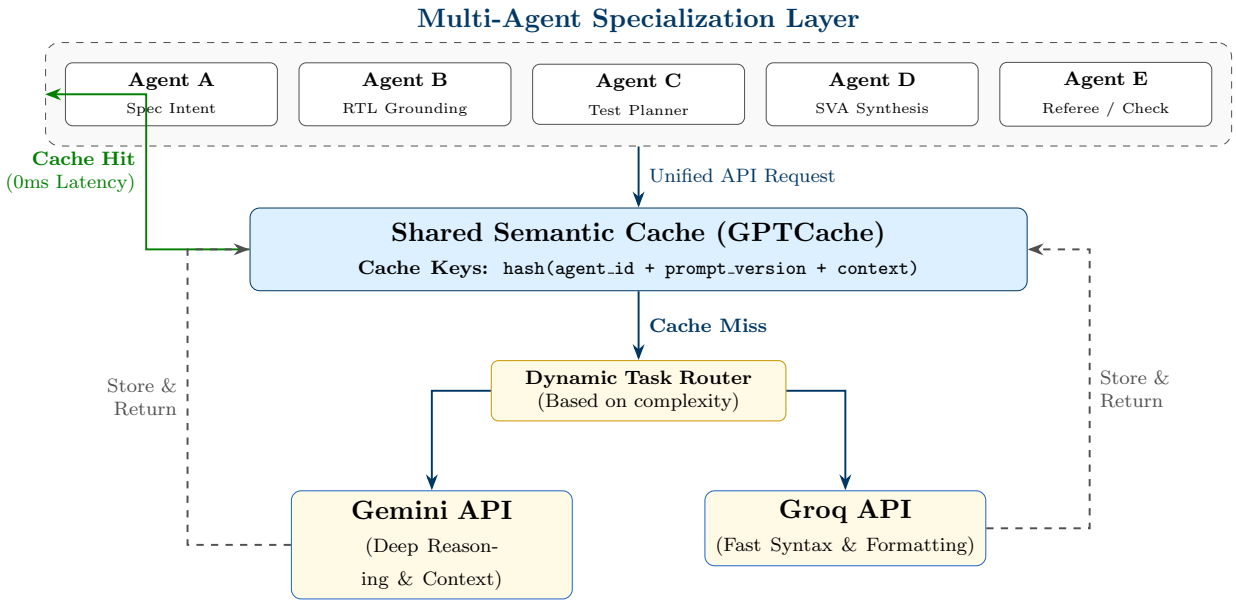


Figure 13: Proposed Multiagent Architecture relying on a unified GPTCache to deduplicate redundant API calls and route tasks efficiently between Gemini (reasoning) and Groq (syntax).

5.4.1 Agent Specialization Roles

To address **RQ2.1 (Coordinating agents to avoid overlap while preserving diversity)**, the SVA generation workload is strictly decomposed among the following dedicated agents. This separation of concerns ensures zero task duplication, while Agent E provides diverse, adversarial oversight:

- **Agent A (Spec Understanding):** Extracts requirement-level semantics, constraints, and edge-case behavior from natural language.
- **Agent B (RTL Grounding):** Maps textual semantics directly to structural RTL nodes and parsed signals (integrating the Signal Mapping pipeline).
- **Agent C (Assertion Planner):** Converts grounded requirements into human-readable, test-plan style verification intents.
- **Agent D (SVA Synthesizer & Repair):** Generates the formal SystemVerilog Assertion syntax and iteratively repairs it based on compiler feedback.
- **Agent E (Referee):** Cross-checks the final, syntactically valid SVAs against the original specification intent to prevent over-constraining.

5.4.2 Model Strategy and Cache Integration

Addressing **RQ2.3 (Task allocation strategies for optimal quality/cost trade-offs)**, a dynamic router intercepts all cache misses and directs them based on task characteristics. Heavy reasoning tasks requiring massive context windows (Agents A and E) are routed to the **Gemini API** (maximizing quality), while high-speed, structured formatting tasks (Agents C and D) are routed to the **Groq API** (minimizing latency and cost).

To ensure system stability, all API interactions pass through a unified response envelope that standardizes output formats across different models. This envelope tracks:

- `content`: The actual generated text or code.
- `model_name`: The backend utilized (e.g., `gemini-1.5-pro` or `llama3-70b-8192`).
- `cache_hit`: Boolean flag indicating if computation was bypassed.
- `latency_ms` & `token_usage`: Telemetry for final reporting.

5.4.3 Risks and Mitigations

Implementing a shared cache introduces the risk of *cross-agent contamination* or *stale data retrieval*. We mitigate this—and concurrently solve **RQ1.9 (Maintaining secure server-side state for multi-agent access)**—through a strict cryptographic keying strategy. The GPT-Cache key is generated as a hash of the target `agent_id`, the `prompt_version`, and the exact `context` string, ensuring isolated yet globally accessible state retrieval.

By employing this architecture in our final evaluation, we aim to demonstrate a measurable increase in assertion acceptance rates alongside a drastic reduction in average latency and API token consumption.

5.5 Stage 4: Two-Phase Assertion Quality Gate and Repair (P5)

To enable robust, feedback-driven refinement (P5), we design a **Two-Phase Quality Gate** acting as a standalone batch filter (often referred to as TC3). Because LLMs are prone to both semantic hallucinations (inventing non-existent signals) and syntactic errors (violating SystemVerilog language rules), validation must be decoupled into two distinct checkpoints.

This stage intercepts flawed SystemVerilog Assertions (SVAs) and subjects them to an iterative repair loop before they are accepted for downstream formal verification.

5.5.1 Phase 1 (Loop A): Semantic Alias Validation

The first phase acts as a strict semantic pre-filter. A common failure mode in LLM-driven verification is the generation of assertions that use plausible but incorrect signal names (e.g., using `reset` instead of `rst_n`).

- **Identifier Extraction:** The system parses the raw LLM-generated SVA to extract all referenced candidate signal identifiers.
- **Lookup Cross-Referencing:** These identifiers are rigorously compared against the `alias_lookup_table` generated during the Stage 1 Signal Mapping pipeline.
- **Repair Trigger:** If the SVA contains unregistered or hallucinated identifiers, the system triggers a repair prompt, explicitly listing the invalid names and providing the allowed canonical names for substitution.

- **Production Default:** Interestingly, our empirical testing revealed that this strict pre-filter tends to over-reject on noisy or highly generated SVA corpora, as it is highly sensitive to tokenization artifacts. Consequently, Phase 1 is *disabled by default* (`--enable-loop-a`) to favor Phase 2, but remains available for strict-compliance runs.

5.5.2 Phase 2 (Loop B): Verible Syntax Validation

The second phase integrates **Verible** [7] as an automated syntax validation backend. Unlike generic Verilog parsers, Verible natively supports advanced SystemVerilog assertion constructs.

- **Module Wrapping:** Because Verible requires valid contextual syntax to evaluate assertions, the system automatically wraps the bare LLM-generated SVA into a minimal, valid SystemVerilog module, dynamically injecting the necessary RTL port declarations based on the Knowledge Graph.
- **Robust Binary Resolution:** To ensure cross-platform reliability without throwing "not found in PATH" errors, the system utilizes a priority-based local binary resolution chain (`--verible-bin` → system `PATH` → local `.tools/verible/**/bin/`).
- **Feedback Signals (RQ3.1):** If the `verible-verilog-syntax` check fails, it provides highly structured feedback. This includes a binary failure signal (#SynC), `stderr` diagnostic outputs (e.g., missing clocking event, invalid operator), and localized line-level error reporting.

5.5.3 Iterative Feedback-Driven Refinement Mechanism (RQ3.2)

Errors detected in either Phase 1 or Phase 2 trigger a closed-loop refinement cycle. The feedback is converted into improved model behavior through a structured **repair prompt**:

1. Extract the specific error message (alias mismatch or Verible `stderr`) and the failing snippet.
2. Construct a repair prompt containing: the original faulty SVA, the precise error diagnostics, and the relevant RTL context/alias list.
3. Re-invoke the Agent D (SVA Synthesizer) LLM, instructing it to correct the logical or syntactic flaw.

This mechanism forces the LLM to refine its tool-use strategy, learning to satisfy both structural hardware realities and rigid compiler constraints.

5.5.4 Stability, Safeguards, and Traceability (RQ3.3)

To ensure that refinement improves performance without causing infinite loops or global model drift, we enforce strict boundary conditions:

- **Bounded Iterations:** The repair loop is capped at a strict maximum number of attempts per assertion to prevent API cost overruns and overfitting.
- **Context Preservation:** The original design specification and structural RTL intent are preserved in the prompt across iterations, ensuring the SVA does not drift away from functional correctness just to satisfy the syntax checker.
- **Artifact Traceability:** The pipeline categorizes all outputs deterministically into `accepted/` and `discarded/` directories. Detailed diagnostics are logged as first-class artifacts, including `reports/summary.json`, `detailed_results.json`, and iteration-level repair logs.

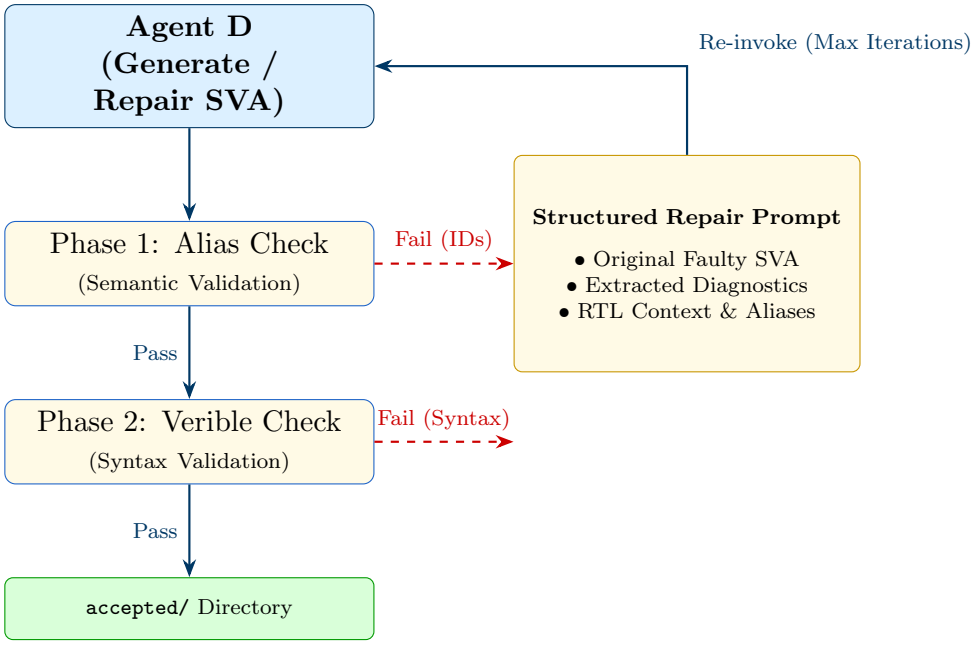


Figure 14: The closed-loop Two-Phase Quality Gate. Assertions must pass both semantic identifier checks and SystemVerilog syntax checks. Failures trigger highly specific diagnostic prompts back to the generation agent.


```
12    {
13      "file_name": "bad_missing_endproperty.sva",
14      "status": "discarded",
15      "final_sva": "property p_bad3;\n  @(posedge PCLK) (PSEL) |-> ##1 (PREADY);\nassert property (p_bad3);",
16      "loop_a_repaired": false,
17      "loop_b_repaired": true,
18      "reason": "Loop B failed after 1 repair attempt (Verible syntax still invalid)\nFinal Verible error:\n/var/folders/gf
19      "invalid_signals_before_repair": [],
20      "syntax_error_before_repair": "/var/folders/gf/4trhcx9n3036fwzjjkf7gk1h0000gn/T/tmpibnvtqfh.sv:3:3: syntax error at t
21    },
22    {
23      "file_name": "bad_missing_semicolon.sva",
24      "status": "accepted",
25      "final_sva": "property p_bad1;\n  @(posedge PCLK) (PSEL && PENABLE) |-> (PRDATA == PWDATA)\nendproperty\nassert prope
26      "loop_a_repaired": false,
27      "loop_b_repaired": false,
28      "reason": "Accepted (passed Loop A + Loop B)",
29      "invalid_signals_before_repair": [],
30      "syntax_error_before_repair": ""
31    },
```


Figure 15: Output logs from the Verible syntax validation phase, highlighting the structured error diagnostics utilized for LLM repair prompting.

5.5.5 Summary

The Two-Phase Quality Gate provides a lightweight, highly automated feedback signal that enables closed-loop refinement of LLM-generated SVAs. By isolating semantic hallucination checks from rigorous syntax validation, and transforming localized compiler errors into structured repair prompts, the system incrementally improves assertion validity while maintaining absolute traceability.

6. Conclusion

In this project, we successfully addressed the critical bottlenecks of LLM-driven SystemVerilog Assertion (SVA) generation. Moving beyond the limitations of baseline systems that analyze specifications and RTL in isolation, we designed and implemented a unified, end-to-end framework capable of jointly reasoning over both domains.

Through our Phase II implementation, we systematically resolved the five core challenges (P1–P5) outlined in the problem statement. We bridged the semantic gap between high-level intent and low-level hardware via structured Knowledge Graph extraction and robust Signal Mapping. To overcome long-context limitations and cloud communication overhead, we integrated multi-resolution retrieval (SSR + GRW-AS) with LLMLingua prompt compression, significantly reducing token consumption while preserving critical causal logic. Furthermore, our proposed multi-agent architecture, orchestrated through a shared GPTCache, successfully eliminated redundant API calls and optimized task routing between deep-reasoning (Gemini) and fast-formatting (Groq) models.

Finally, we closed the generation loop by deploying a two-phase quality gate, utilizing Verible to provide automated, localized feedback for iterative syntax repair. By combining these advanced methodologies—structured memory, semantic alignment, aggressive compression, and closed-loop refinement—our pipeline demonstrates a highly scalable, cost-efficient, and structurally aware approach, advancing the reliability and practical viability of LLM-assisted formal verification.

References

[1] Y. Bai, G. Bany Hamad, S. Suhaib, and H. Ren, "AssertionForge: Enhancing Formal Verification Assertion Generation with Structured Representation of Specifications and RTL," *arXiv:2503.19174*, 2025.
https://arxiv.org/abs/2503.19174 — https://github.com/NVlabs/AssertionForge

[2] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," *arXiv:2310.05736*, 2023.
https://arxiv.org/abs/2310.05736

[3] F. Mu, L. Shi, S. Wang *et al.*, "ClarifyGPT: Empowering LLM-based Code Generation with Intention Clarification," *arXiv:2310.10996*, 2023.
https://arxiv.org/abs/2310.10996

[4] W. Fang, M. Li, M. Li *et al.*, "AssertLLM: Generating and Evaluating Hardware Verification Assertions from Design Specifications via Multi-LLMs," *arXiv:2402.00386*, 2024.
https://arxiv.org/abs/2402.00386

[5] D. Edge, H. Trinh, N. Cheng *et al.*, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *arXiv:2404.16130*, 2024.
https://arxiv.org/abs/2404.16130

[6] S. Takamaeda-Yamazaki, "Pyverilog: A Python-based hardware design processing toolkit for Verilog HDL," *Applied Reconfigurable Computing*, Springer, 2015.
https://github.com/PyHDI/Pyverilog

[7] Google/ChipsAlliance, "Verible: SystemVerilog Developer Tools," https://github.com/chipsalliance/verible

[8] "SANGAM: Monte Carlo Tree Search for Iterative SVA Generation," *arXiv:2506.13983*, 2025.
https://arxiv.org/abs/2506.13983

[9] "STELLAR: Structure-Guided LLM SVA Generation," *arXiv:2601.19903*, 2025.
https://arxiv.org/abs/2601.19903

[10] "AssertFix: Fixing SVAs using CDFG-based Context Retrieval and CoT," *arXiv:2509.23972*, 2025.
https://arxiv.org/abs/2509.23972

[11] M. Kang, M. Liu, G. Bany Hamad, S. Suhaib, and H. Ren, "FVEval: Understanding Language Model Capabilities in Formal Verification of Digital Hardware," *arXiv:2410.23299*, 2024.
https://arxiv.org/abs/2410.23299

[12] K. Maddala, B. Mali, and C. Karfa, "LAAG-RV: LLM Assisted Assertion Generation for RTL Design Verification," *IEEE ITC India*, 2024.
https://arxiv.org/abs/2409.15281

[13] Zilliz AI Team, "GPTCache: An Open-Source Semantic Cache for LLM Applications," 2023.
https://github.com/zilliztech/GPTCache